

lab1

February 22, 2025

1 Spark Cluster on GCP

1.1 Introduction

Big data processing has become an essential aspect of modern data-driven applications. Organizations handle vast amounts of structured and unstructured data, requiring powerful distributed computing frameworks for efficient processing. **Apache Spark**, an open-source distributed computing system, is widely used for large-scale data processing due to its in-memory computation capabilities and scalability.

Google Cloud Platform (GCP) offers **Dataproc**, a fully managed service that simplifies running **Apache Spark, Hadoop, and other big data frameworks** on the cloud. **Dataproc enables users to create, manage, and scale Spark clusters effortlessly, reducing infrastructure overhead and improving efficiency.** This assignment focuses on deploying and managing an Apache Spark cluster using **Google Cloud Dataproc**, leveraging both the **Google Cloud Console (GUI-based approach)** and the **gcloud CLI (command-line interface)**.

By following the steps outlined in this assignment, we gain hands-on experience in **creating, running, and optimizing Spark jobs on a cloud-based distributed environment.** The practical knowledge gained from this exercise is valuable for data engineers, data scientists, and cloud architects looking to optimize big data workflows on GCP.

1.2 Objectives

The primary objective of this assignment is to develop a **deep understanding of deploying and managing Spark clusters** on Google Cloud Platform. By completing the tasks, we aim to achieve the following:

1. **Deploy a Dataproc Cluster on GCP**
 - Set up a Spark cluster using **Google Cloud Dataproc**.
 - Configure cluster settings, including **machine types, networking, and storage options**.
2. **Run a Spark Job on Dataproc**
 - Execute a **basic Spark job** to process and analyze data.
 - Monitor and validate the execution of the job using **logs and performance metrics**.
3. **Verify and Analyze Job Output**
 - Access and interpret Spark job results stored in **Google Cloud Storage (GCS)**.
 - Understand the performance and behavior of Spark on a cloud-based cluster.
4. **Modify Cluster Configuration**

- Scale the cluster by **adding or removing worker nodes** to optimize performance.
 - Assess the impact of cluster scaling on **job execution time and resource utilization**.
5. **Compare Console vs. CLI-Based Management**
 - Perform all cluster operations using both:
 - **Google Cloud Console (Graphical UI)**
 - **gcloud CLI (Command-Line Interface)**
 - Compare ease of use, flexibility, and automation capabilities.
 6. **Optimize Job Performance (Optional)**
 - Explore **tuning parameters** such as executor memory, parallelism, and caching.
 - Optimize Spark jobs for **faster execution and lower cost**.
 7. **Clean Up Cloud Resources**
 - Shut down or delete the Dataproc cluster after completing tasks to avoid unnecessary cloud charges.
 - Ensure proper cleanup of **temporary storage and logs**.
 8. **Document the Process and Findings**
 - Provide a **structured report** detailing the steps taken, observations, and challenges encountered.
 - Include **screenshots, log outputs, and performance insights** to support the findings.

1.3 Methodology

1.4 Deploying an Apache Spark Cluster on Google Cloud Console

This section outlines the step-by-step process followed to create an Apache Spark cluster on **Google Cloud Dataproc**, submit a job, and attempt to scale the cluster. It also details the encountered challenges related to **insufficient SSD quota** while increasing the number of worker nodes.

1.4.1 Step 1: Creating a Dataproc Cluster Using Google Cloud Console

To set up the Spark cluster, I followed the official **Google Cloud Dataproc Quickstart Guide** available at:

[Creating a Cluster via Console](#)

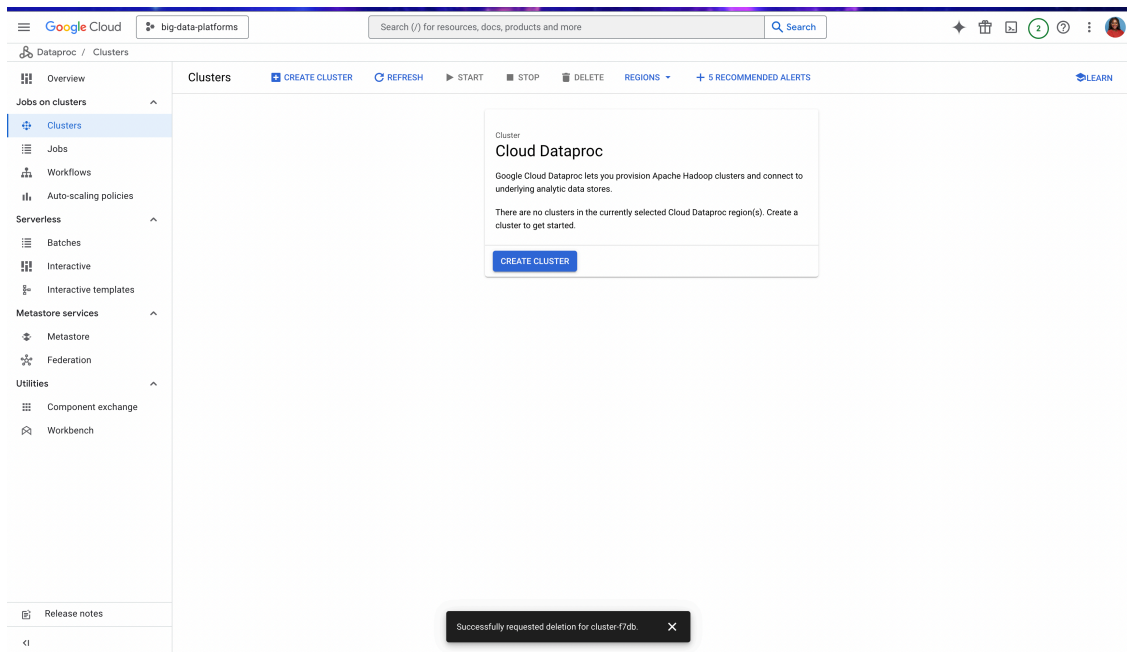
Steps Taken:

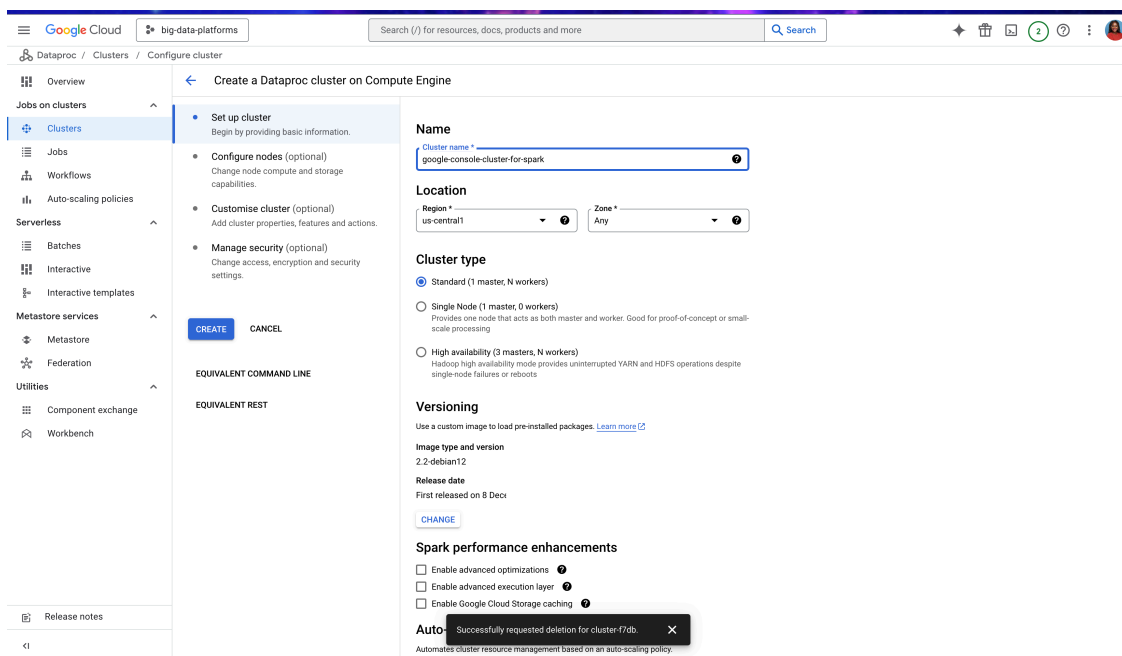
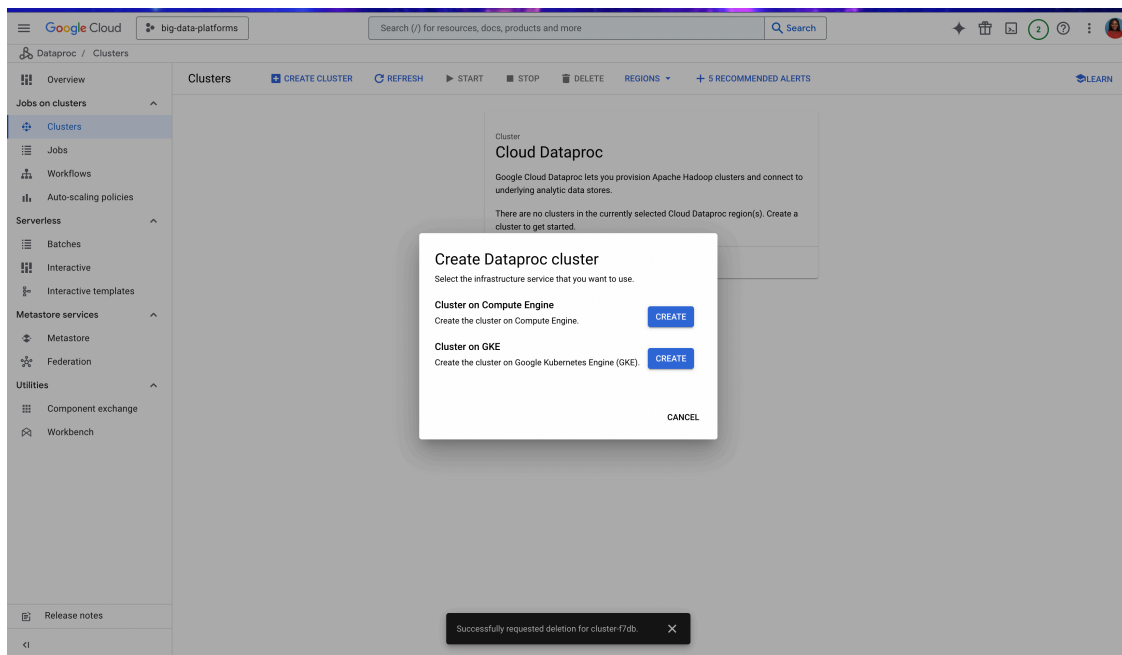
1. **Navigated to Dataproc Service:**
 - Opened the [Google Cloud Console](#).
 - Went to **Navigation Menu → Dataproc → Clusters**.
2. **Created a New Cluster:**
 - Clicked **“Create Cluster”** and selected **Standard Cluster**.
 - Entered cluster name: **google-console-cluster-for-spark**.
 - Set the **Region** to **us-central1**.
 - Set the **Zone** to **us-central1-f**.
3. **Configured Cluster Settings:**
 - **Master Node and Worker Machine Type:** Standard (1 master, N workers).
 - Used the default for the rest of the settings
4. **Created the Cluster:**

- Clicked “**Create**”, and after a few minutes, the cluster was successfully provisioned.

```
[34]: from IPython.display import display, Image

display(Image(filename="screenshots/cd1.png"))
display(Image(filename="screenshots/cd2.png"))
display(Image(filename="screenshots/cd3.png"))
display(Image(filename="screenshots/cd4.png"))
display(Image(filename="screenshots/cd5.png"))
display(Image(filename="screenshots/cd6.png"))
display(Image(filename="screenshots/cd7.png"))
display(Image(filename="screenshots/cd8.png"))
```





Google Cloud

big-data-platforms

Search (/) for resources, docs, products and more

Search

Dataproc / Clusters / Configure cluster

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Component exchange

Workbench

Release notes

← Create a Dataproc cluster on Compute Engine

Set up cluster

Begin by providing basic information.

Configure nodes (optional)

Change node compute and storage capabilities.

Customise cluster (optional)

Add cluster properties, features and actions.

Manage security (optional)

Change access, encryption and security settings.

CREATE

CANCEL

EQUIVALENT COMMAND LINE

EQUIVALENT REST

worker nodes

Each contains a YARN NodeManager and a HDFS DataNode. HDFS replication factor is 2.

General purpose

Compute-optimised

Memory-optimised

GPUs

Machine types for common workloads, optimised for cost and flexibility

Series

N2

Powered by Intel Cascade Lake and Ice Lake CPU platforms

Machine type

n2-standard-4 (4 vCPU, 2 core, 16 GB memory)

vCPU

4

Memory

3.75 GB

CPU PLATFORM AND GPU

Number of worker nodes *

2

Primary disk size *

200

GB

Primary disk type *

Balanced Persistent Disk

Number of local SSDs *

0

x 375GB

Local SSD interface

SCSI

Secondary worker nodes

Each contains a YARN NodeManager. HDFS does not run on secondary worker nodes. Secondary worker VMs are preemptible by default. Spot and pre-emptible VMs cost less, but can be terminated at any time due to system demands. [Learn more](#)

Sole tenancy

Enable to create this cluster on sole-tenant nodes. This grants exclusive access to a physical Compute Engine server that is dedicated to hosting only your cluster's VMs. If you are creating a cluster in a region that uses an [Sole tenancy](#) VM, you must also [enable](#) it.

Successfully requested deletion for cluster-f7db.

×

Enable

Google Cloud

big-data-platforms

Search (/) for resources, docs, products and more

Search

Dataproc / Clusters / Configure cluster

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Component exchange

Workbench

Release notes

← Create a Dataproc cluster on Compute Engine

Set up cluster

Begin by providing basic information.

Configure nodes (optional)

Change node compute and storage capabilities.

Customise cluster (optional)

Add cluster properties, features and actions.

Manage security (optional)

Change access, encryption and security settings.

CREATE

CANCEL

EQUIVALENT COMMAND LINE

EQUIVALENT REST

Internal IP only

Configure all instances to have only internal IP addresses. [Learn more](#)

Labels

A list of key-value pairs to attach to the cluster for tracking.

+ ADD LABELS

Cluster properties

Use cluster properties to add or modify configuration files when creating a cluster.

+ ADD PROPERTIES

Initialisation actions

Use initialisation actions to customise settings, install applications or make other modifications to your cluster. Select scripts or executables that Cloud Dataproc will run when provisioning your cluster.

+ ADD INITIALISATION ACTION

Custom cluster metadata

Add custom metadata to cluster instances. [Learn more](#)

+ ADD METADATA

Scheduled deletion

Use scheduled deletion to help avoid incurring Google Cloud charges for an inactive cluster. [Learn more](#)

Delete on a fixed time schedule

Delete after a cluster idle time period without submitted jobs

Cloud Storage staging bucket

5

Google Cloud

big-data-platforms

Search (/) for resources, docs, products and more

Search

Dataproc / Clusters / Configure cluster

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Component exchange

Workbench

Release notes

← Create a Dataproc cluster on Compute Engine

Set up cluster

Begin by providing basic information.

Configure nodes (optional)

Change node compute and storage capabilities.

Customise cluster (optional)

Add cluster properties, features and actions.

Manage security (optional)

Change access, encryption and security settings.

CREATE

CANCEL

EQUIVALENT COMMAND LINE

EQUIVALENT REST

Project access

☐ Enables the cloud platform scope for this cluster [Learn more](#)

Encryption

Encrypt cluster persistent disk data and optionally job argument data. [Learn more](#)

☒ Google-managed encryption key
Keys owned by Google

☐ Cloud KMS key
Keys owned by customers

☐ Enable Confidential Computing
Confidential Computing on clusters can only be enabled if all nodes on the cluster use the N2D machine type. [Learn more](#)

Personal cluster authentication

Enable Dataproc personal cluster authentication to allow interactive workloads on the cluster to securely run as your end user identity. [Learn more](#)

☒ Enable

Secure Multi-Tenancy

Enable Dataproc Service Account-Based Secure Multi-tenancy to share a cluster with multiple users. Make sure the VM service account for the cluster has the proper permissions to impersonate all mapped service accounts for users. [Learn more](#)

☒ Enable

Kerberos and Hadoop Secure Mode

Enable Kerberos and Hadoop Secure Mode to provide user authentication, isolation and encryption within a Dataproc cluster. [Learn more](#)

☒ Enable

Google Cloud

big-data-platforms

Search (/) for resources, docs, products and more

Search

Dataproc / Clusters

Overview

Jobs on clusters

Clusters

Jobs

Workflows

Auto-scaling policies

Serverless

Batches

Interactive

Interactive templates

Metastore services

Metastore

Federation

Utilities

Component exchange

Workbench

Release notes

Clusters

CREATE CLUSTER

REFRESH

START

STOP

DELETE

REGIONS

+ 5 RECOMMENDED ALERTS

Filter

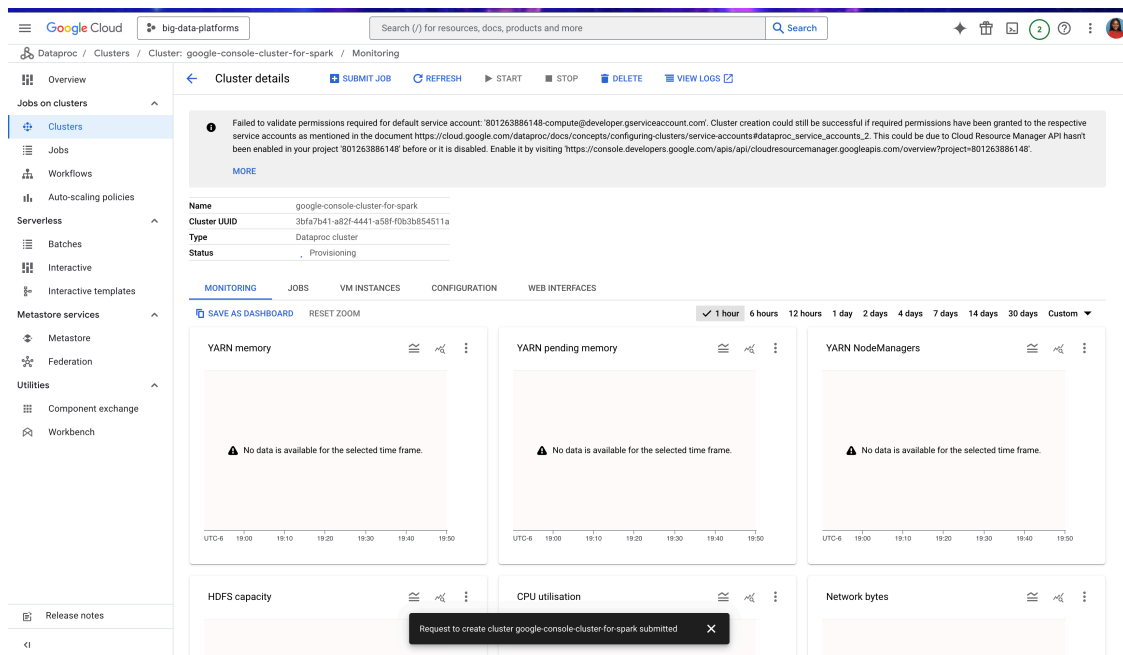
Search cluster by properties, press Enter

Sorry, the server was not able to fulfil your request.

☐	Name ↑	Status	Region	Zone	Total worker nodes	Flexible VMs?	Scheduled deletion	Cloud Storage staging bucket	Created	Labels
☐	google-console-cluster-for-spark	Provisioning	us-central1	us-central1-f	2	No	Off	dataproc-staging-us-central1-801263886148-kg5dztgw	21 Feb 2025, 19:49:54	goog-dataproc:enabled

Request to create cluster google-console-cluster-for-spark submitted

6



1.4.2 Step 2: Submitting a Spark Job

Once the cluster was running, I submitted a **Spark job** to process data.

Steps Taken:

1. Navigated to Dataproc Jobs:
 - In Google Cloud Console → Dataproc → Jobs, clicked “Submit Job”.
2. Configured the Job:
 - **Job Type:** Spark
 - **Main Class:** `org.apache.spark.examples.SparkPi`
 - **Jar File Location:**
`file:///usr/lib/spark/examples/jars/spark-examples.jar`
 - **Arguments (Input & Output Paths):**
`1000`
 - **Selected Cluster:** `google-console-cluster-for-spark`.
3. Submitted the Job:
 - Clicked **Submit**, and the job ran successfully.
 - Checked **logs and execution details** to verify that the job completed without errors.

```
[35]: display(Image(filename="screenshots/cd9.png"))
display(Image(filename="screenshots/cd10.png"))
```

The screenshot shows the Google Cloud Dataproc Clusters page. The left sidebar contains navigation links for Overview, Clusters, Jobs, Workflows, Auto-scaling policies, Serverless, Batches, Interactive, Interactive templates, Metastore services, Metastore, Federation, Utilities, Component exchange, and Workbench. The main panel displays the 'Cluster details' for 'google-console-cluster-for-spark'. A message indicates a failure to validate permissions for the default service account. Below this, there are tabs for MONITORING, JOBS, VM INSTANCES, CONFIGURATION, and WEB INTERFACES. The MONITORING tab shows charts for YARN memory, YARN pending memory, HDFS capacity, and CPU utilisation, all with a warning that no data is available for the selected time frame. On the right, the 'Submit a job' dialog is open, showing fields for Job ID (job-f0f4d527), Job type (Spark), Main class or JAR (org.apache.spark.examples.SparkPi), Jar files (file:///usr/lib/spark/examples/jars/spark-examples.jar), Files, Archive files, Arguments (1000), and checkboxes for advanced optimizations and execution layer.

The screenshot shows the Google Cloud Dataproc Jobs page. The left sidebar is the same as the previous image. The main panel displays the 'Job details' for 'job-f0f4d527'. The job status is 'Succeeded'. Below the details, there are tabs for MONITORING and CONFIGURATION. The MONITORING tab shows charts for YARN memory, YARN pending memory, and YARN NodeManagers. The OUTPUT tab is selected, showing the job's output logs. The logs indicate that the job completed successfully, with a message 'Spark jobs take ~60 seconds to initialise resources.' and 'Output is complete'. At the bottom, there is a section for 'EQUIVALENT COMMAND LINE'.

1.4.3 Step 3: Attempting to Scale the Cluster (Increasing Worker Nodes)

After running the job, I attempted to **increase the number of worker nodes** to analyze performance improvements.

Steps Taken:

1. Went to Cluster Settings:

- Opened **Google Cloud Console** → **Dataproc** → **Clusters**.
- Selected **google-console-cluster-for-spark**.
- Clicked **“Edit”** to modify the cluster configuration.

2. Updated Worker Nodes:

- Increased **Worker Node Count** from **2** to **5**.

3. Error Encountered:

- The operation failed with the following error:

Failed to update cluster google-console-cluster-for-spark
Insufficient 'SSD_TOTAL_GB' quota. Requested 600.0, available 0.0.

- This means my project had reached the **maximum SSD quota (500GB)** and could not allocate more storage.

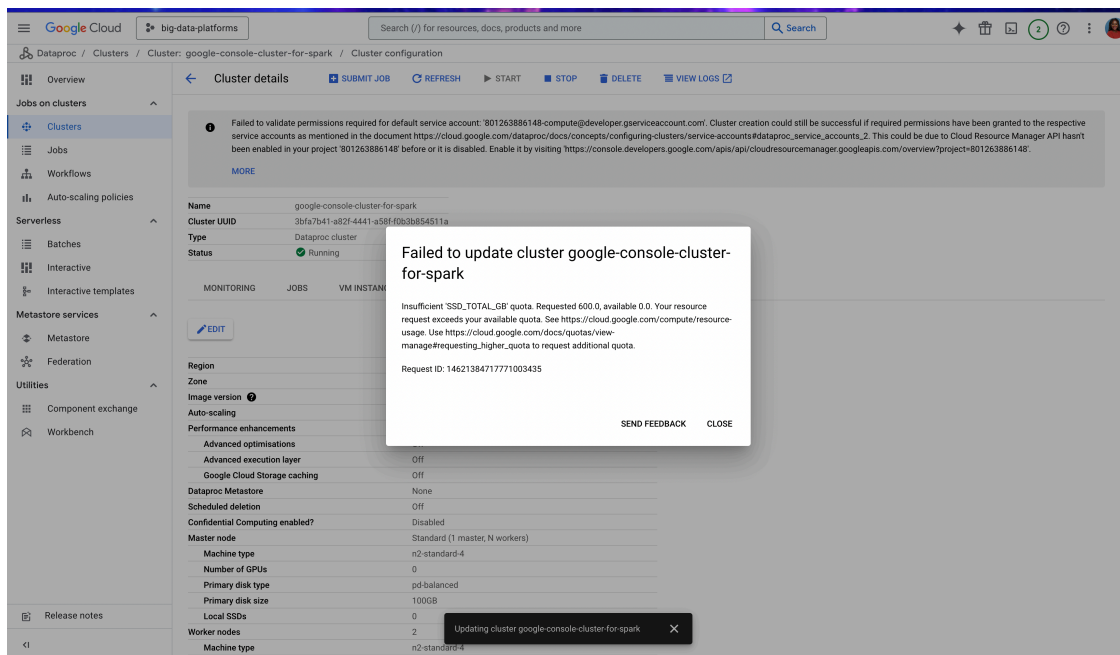
4. Deleted Cluster and Created New One:

- Created new cluster with Machine type **n2-standard-2** (2 vCPU, 1 core, 8 GB memory) instead of **n2-standard-4** (4 vCPU, 2 core, 16 GB memory)
- Repeated steps 1 - 3, still the same error

```
[36]: display(Image(filename="screenshots/cd11.png"))  
display(Image(filename="screenshots/cd12.png"))
```

The screenshot displays the Google Cloud Console interface for editing a Dataproc cluster. The main panel shows the 'CONFIGURATION' tab with various settings. The 'Worker nodes' field is set to 5, and the 'Secondary worker nodes' field is set to 0. The 'Labels' section lists five keys with their corresponding values. The 'SAVE' button is highlighted in blue. The background shows the 'Cluster details' page with a warning message about permissions.

Key	Value
Key 1	goog-dataproc-cluster-name
Key 2	goog-dataproc-cluster-uid
Key 3	goog-dataproc-location
Key 4	goog-drz-dataproc-uid
Key 5	goog-dataproc-autozone



1.4.4 Step 4: Investigating the SSD Quota Issue

To understand the issue, I checked my **current SSD quota**.

Steps Taken:

1. **Checked Quotas in Google Cloud Console:**
 - Opened Navigation Menu → IAM & Admin → Quotas.
 - Searched for “SSD_TOTAL_GB”.
 - Found that my quota was **500GB (Max)**, with **500GB** already in use.
2. **Attempted to Request a Higher Quota:**
 - Tried to **increase SSD quota** through the **Manage Quotas** page.
 - Google Cloud **did not allow increasing SSD storage above 500GB**.

```
[37]: display(Image(filename="screenshots/cd15.png"))
display(Image(filename="screenshots/cd13.png"))
display(Image(filename="screenshots/cd14.png"))
```


Google Cloud console showing Quotas and system limits for project 'big-data-platforms'.

Quotas and system limits for project 'big-data-platforms'

Values for quotas are being updated. This may take two to three weeks to complete.

Current usage > 90%
1

All quotas and system limits
19,353

Service	Name	Type	Dimensions (e.g. location)	Value	Current usage percentage	Current usage
Compute Engine API	Persistent Disk SSD (GB)	Quota	region: us-central1	500 GB	100%	500 GB
Compute Engine API	CPUs (all regions)	Quota		32	25%	8
Compute Engine API	Networks	Quota		5	20%	1
Compute Engine API	N2 CPUs	Quota	region: us-central1	48	8.33%	4
Compute Engine API	VM instances	Quota	region: us-central1	24	8.33%	2
Compute Engine API	Subnetwork ranges per VPC Network	Quota	network_id: default	500	8.2%	41
Compute Engine API	Firewall rules	Quota		100	4%	4
Cloud Dataproc API	Requests involving a cluster	Quota	region: us-central1	200	1.25%	2.5

Now viewing project 'big-data-platforms' in organisation 'No organisation'

Google Cloud console showing the 'Create a Dataproc cluster on Compute Engine' wizard.

Create a Dataproc cluster on Compute Engine

Set up cluster
Begin by providing basic information.

- Configure nodes (optional)**
Change node compute and storage capabilities.
- Customise cluster (optional)**
Add cluster properties, features and actions.
- Manage security (optional)**
Change access, encryption and security settings.

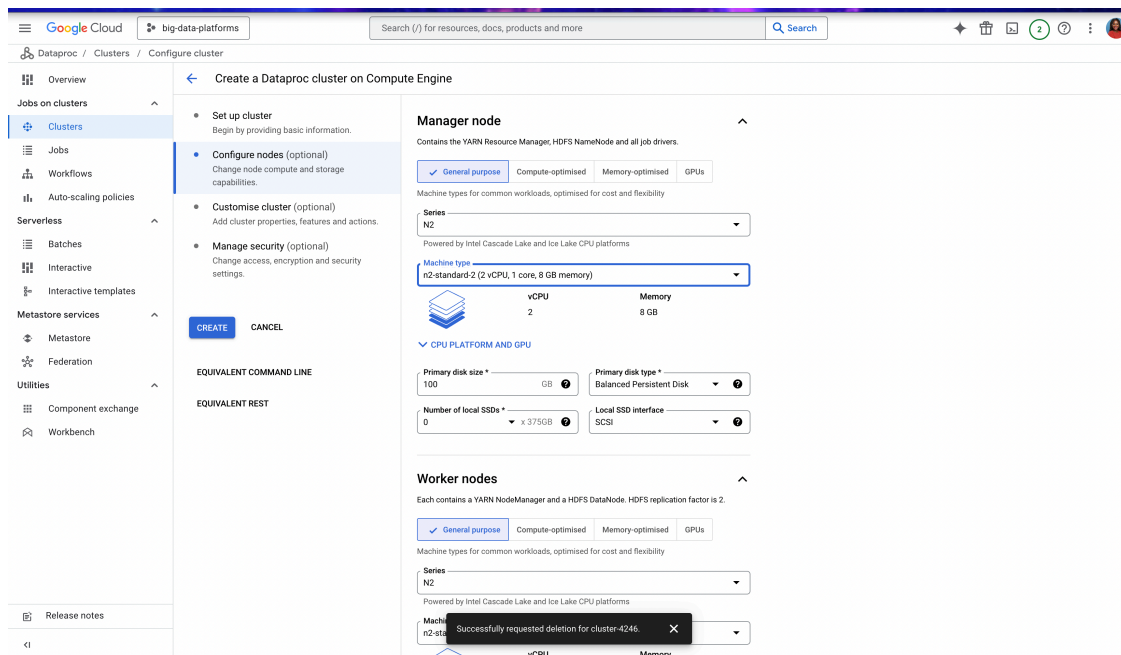
Location
Region: us-central1, Zone: Any

Cluster type
Standard (1 master, N workers)

Versioning
Image type and version: 2.2-debian12
Release date: First released on 8 Dec

Spark performance enhancements
☐ Enable advanced optimizations
☐ Enable advanced execution layer
☐ Enable Google Cloud Storage caching

Auto
Successfully requested deletion for cluster-4246.
Automates cluster resource management based on an auto-scaling policy.



1.5 Step 5: Workaround and Next Steps

Since the SSD quota could not be increased, I considered alternative solutions:

1.5.1 Possible Workarounds:

Use standard persistent disks (pd-standard) instead of SSD (pd-ssd)

Reduce the number of worker nodes to stay within the quota limit

Create a new cluster with standard persistent disks

To proceed further, I need to decide whether to: 1. **Modify the cluster to use standard persistent disks instead of SSDs.** 2. **Reduce the number of worker nodes to fit within the available quota.**

1.5.2 Step 6: Deleted Dataproc Cluster

Once I was done I deleted the running cluster.

Steps Taken:

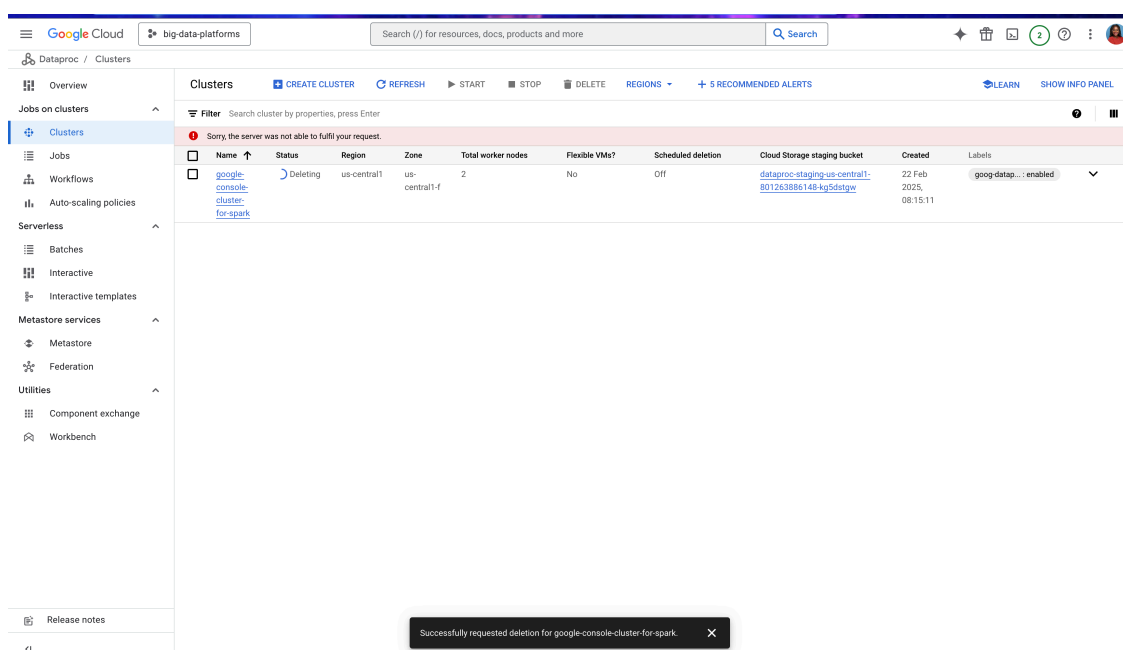
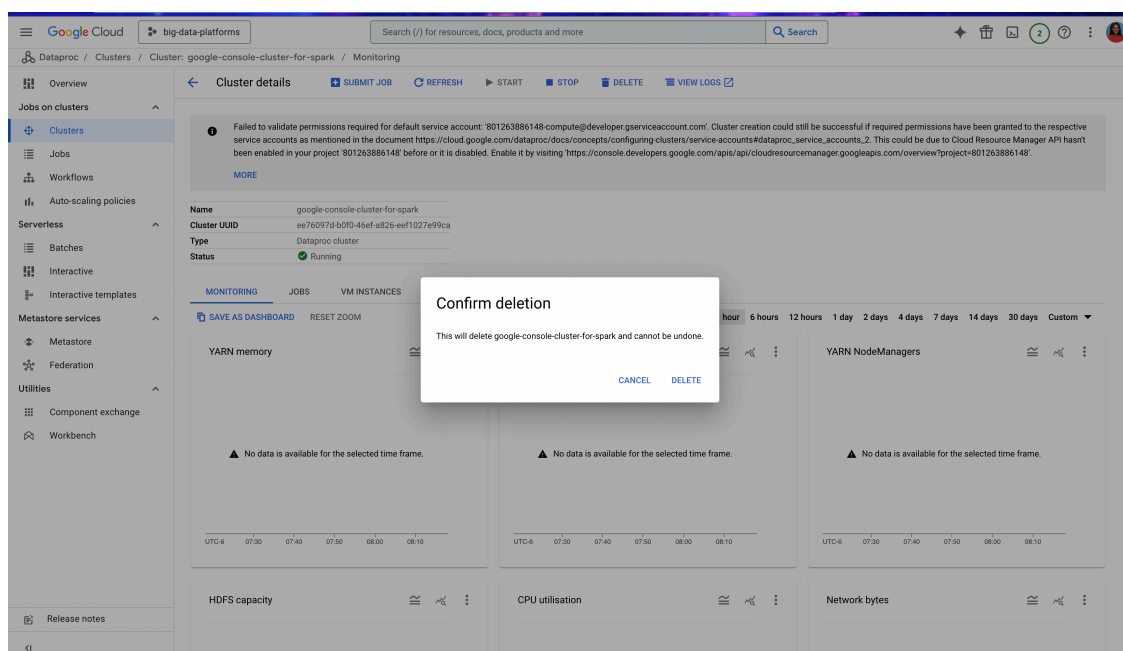
1. Navigated to Clusters:

- In Google Cloud Console → Dataproc → Delete, clicked “Delete”.

2. Confirmed the Delete:

- **Deletion Job Started:** Waited a couple of minutes till deletion was over and back to the create cluster view.

```
[38]: display(Image(filename="screenshots/cd16.png"))
display(Image(filename="screenshots/cd17.png"))
```

1.5.3 Summary

Task	Status	Notes
Created Dataproc Cluster	Success	Used 2 worker nodes

Task	Status	Notes
Submitted Spark Job	Success	Job completed without errors
Increased Worker Nodes	Failed	Insufficient SSD quota (500GB limit)
Checked SSD Quota	Success	500GB max, already fully used
Requested More Quota	Not Allowed	Could not increase beyond 500GB
Deleted Cluster	Success	Cluster got deleted without errors

1.6 Deploying an Apache Spark Cluster on Google Cloud Dataproc (CLI)

This report outlines the step-by-step process of **creating an Apache Spark cluster** using **Google Cloud Dataproc CLI**, submitting a job, and **attempting to scale the cluster**. The report also details the **challenges faced with SSD storage quotas**, attempts to increase quotas, and the final resolution.

1.6.1 Step 1: Creating a Dataproc Cluster with SSD Storage (CLI)

To set up the cluster, I used the `gcloud` command to **create a Dataproc cluster with SSD storage (pd-ssd)**.

Command Used:

```
gcloud dataproc clusters create google-cli-cluster-for-spark \
  --enable-component-gateway \
  --region=us-central1 \
  --master-machine-type n2-standard-4 \
  --master-boot-disk-type pd-ssd \
  --master-boot-disk-size 100 \
  --num-workers 2 \
  --worker-machine-type n2-standard-4 \
  --worker-boot-disk-type pd-ssd \
  --worker-boot-disk-size 200 \
  --image-version 2.2-debian12 \
  --project big-data-platforms-44
```

Response: Cluster Created Successfully

Cluster `google-cli-cluster-for-spark` was successfully provisioned and placed in `us-central1-f`.

[39]: `display(Image(filename="screenshots/cd18.png"))`

```
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters list --project=big-data-platforms-44 --region=us-central1
```

NAME	PLATFORM	PRIMARY_WORKER_COUNT	SECONDARY_WORKER_COUNT	STATUS	ZONE	SCHEDULED_DELETE
cluster-f98e	GCE	2		RUNNING	us-central1-f	

```
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters create google-cli-cluster-for-spark --enable-component-gateway --region=us-central1 --no-address --master-machine-type n2-standard-4 --master-boot-disk-type pd-balanced --master-boot-disk-size 100 --num-workers 2 --worker-machine-type n2-standard-4 --worker-boot-disk-type pd-balanced --worker-boot-disk-size 200 --image-version 2.2-debian12 --project big-data-platforms-44
```

ERROR: (gcloud.dataproc.clusters.create) INVALID_ARGUMENT: Insufficient 'SSD_TOTAL_GB' quota. Requested 500.0, available 0.0. Your resource request exceeds your available quota. See <https://cloud.google.com/compute/resource-usage>. Use https://cloud.google.com/docs/quotas/view-manage#requesting_higher_quota to request additional quota.

```
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters list --project=big-data-platforms-44 --region=us-central1
```

Listed 0 items.

```
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters create google-cli-cluster-for-spark --enable-component-gateway --region=us-central1 --no-address --master-machine-type n2-standard-4 --master-boot-disk-type pd-balanced --master-boot-disk-size 100 --num-workers 2 --worker-machine-type n2-standard-4 --worker-boot-disk-type pd-balanced --worker-boot-disk-size 200 --image-version 2.2-debian12 --project big-data-platforms-44
```

Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/1b5cbac2-eede-3cce-938f-8e4f91289e73].

Waiting for cluster creation operation..."

WARNING: Failed to validate permissions required for default service account: '801263886148-compute@developer.gserviceaccount.com'. Cluster creation could still be successful if required permissions have been granted to the respective service accounts as mentioned in the document https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/service-accounts#dataproc_service_accounts_2. This could be due to Cloud Resource Manager API hasn't been enabled in your project '801263886148' before or it is disabled. Enable it by visiting '<https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview?project=801263886148>'.

WARNING: The firewall rules for specified network or subnetwork would allow ingress traffic from 0.0.0.0/0, which could be a security risk.

WARNING: The specified custom staging bucket 'dataproc-staging-us-central1-801263886148-kg5dstgw' is not using uniform bucket level access IAM configuration. It is recommended to update bucket to enable the same. See <https://cloud.google.com/storage/docs/uniform-bucket-level-access>.

Waiting for cluster creation operation...done.

Created [<https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark>] Cluster placed in zone [us-central1-f].

```
(base) chiomaonyekpere@MacBookPro ~ %
```

1.6.2 Step 2: Submitting a Spark Job (CLI)

After setting up the cluster, I submitted a Spark job to calculate Pi using the built-in SparkPi example.

Command Used:

```
gcloud dataproc jobs submit spark --cluster=google-cli-cluster-for-spark \
  --region=us-central1 \
  --class org.apache.spark.examples.SparkPi \
  --jars file:///usr/lib/spark/examples/jars/spark-examples.jar -- 1000
```

Response: Job Submitted Successfully

The job executed without errors, and the output confirmed the approximate value of Pi.

```
[40]: display(Image(filename="screenshots/cd19.png"))
display(Image(filename="screenshots/cd19-1.png"))
```

```

Updated property [core/project].
(base) chimaonyekpere@MacBookPro ~ % gcloud dataproc jobs submit spark --cluster=google-cli-cluster-for-spark-m-local-10128-0-16-8032
--region=us-central1 \
--class org.apache.spark.examples.SparkPi \
--jars file:///usr/lib/spark/examples/jars/spark-examples.jar -- 1000

Job [c1c5ad6c71ab423f9f3862a12d2e88a1] submitted.
Waiting for job output...
25/02/22 10:39:25 INFO SparkEnv: Registering MapOutputTracker
25/02/22 10:39:25 INFO SparkEnv: Registering BlockManagerMaster
25/02/22 10:39:25 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
25/02/22 10:39:25 INFO SparkEnv: Registering OutputCommitCoordinator
25/02/22 10:39:25 INFO MetricsConfig: Loaded properties from hadoop-metrics2.properties
25/02/22 10:39:25 INFO MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
25/02/22 10:39:25 INFO MetricsSystemImpl: google-hadoop-file-system metrics system started
25/02/22 10:39:26 INFO DataprocSparkPlugin: Registered 188 driver metrics
25/02/22 10:39:26 INFO DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at google-cli-cluster-for-spark-m-local-10128-0-16-8032
25/02/22 10:39:27 INFO AHSProxy: Connecting to Application History server at google-cli-cluster-for-spark-m-local-10128-0-16-10200
25/02/22 10:39:27 INFO Configuration: resource-types.xml not found
25/02/22 10:39:27 INFO ResourceUtils: Unable to find 'resource-types.xml'.
25/02/22 10:39:29 INFO YarnClientImpl: Submitted application application_1740218524166_0001
25/02/22 10:39:30 INFO DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at google-cli-cluster-for-spark-m-local-10128-0-16-8030
25/02/22 10:39:31 INFO GhfsGlobalStorageStatistics: periodic connector metrics: {gcs_api_client_non_found_response_count=1, gcs_api_client_side_error_count=1, gcs_api_time=384, gcs_api_total_request_count=2, gcs_connector_time=482, gcs_list_file_request=1, gcs_list_file_request_duration=191, gcs_list_file_request_max=191, gcs_list_file_request_mean=191, gcs_list_file_request_min=191, gcs_metadata_request=1, gcs_metadata_request_duration=193, gcs_metadata_request_max=193, gcs_metadata_request_mean=193, gcs_metadata_request_min=193, gs_filesystem_initialize=2, op_get_file_status=1, op_get_file_status_duration=482, op_get_file_status_max=482, op_get_file_status_mean=482, op_get_file_status_min=482, uptimeSeconds=7} [CONTEXT ratelimit_period="5 MINUTES" ]
25/02/22 10:39:31 INFO GoogleCloudStorageImpl: Ignoring exception of type GoogleJsonResponseException; verified object already exists with desired state.
25/02/22 10:39:32 INFO GoogleHadoopOutputStream: hflush(): No-op due to rate limit (RateLimiter[stableRate=0.2qps]): readers will *not* yet see flushed data for gs://dataproc-temp-us-central1-801263886148-aggwphvu/47db7827-9aa0-474b-8aa6-a06a043435de/spark-job-history/application_1740218524166_0001.inprogress [CONTEXT ratelimit_period="1 MINUTES" ]
Pi is roughly 3.141545671415457
25/02/22 10:39:48 INFO DataprocSparkPlugin: Shutting down driver plugin. metrics=[action_http_patch_request=0, files_created=1, gcs_

```

```

ead_operations_duration=0, action_http_delete_request_duration=98, action_http_post_request_duration=400, op
Job [c1c5ad6c71ab423f9f3862a12d2e88a1] finished successfully.
done: true
driverControlFilesUri: gs://dataproc-staging-us-central1-801263886148-kg5dstgw/google-cloud-dataproc-metainfo/47db7827-9aa0-474b-8aa6-a06a043435de/jobs/c1c5ad6c71ab423f9f3862a12d2e88a1/
driverOutputResourceUri: gs://dataproc-staging-us-central1-801263886148-kg5dstgw/google-cloud-dataproc-metainfo/47db7827-9aa0-474b-8aa6-a06a043435de/jobs/c1c5ad6c71ab423f9f3862a12d2e88a1/driveroutput
jobUuid: 6cd63719-afcc-33e2-918f-cdd5750be6f8
placement:
  clusterName: google-cli-cluster-for-spark
  clusterUuid: 47db7827-9aa0-474b-8aa6-a06a043435de
reference:
  jobId: c1c5ad6c71ab423f9f3862a12d2e88a1
  projectId: big-data-platforms-44
sparkJob:
  args:
    - '1000'
  jarFileUri:
    - file:///usr/lib/spark/examples/jars/spark-examples.jar
  mainClass: org.apache.spark.examples.SparkPi
status:
  state: DONE
  stateStartTime: '2025-02-22T10:39:51.732552Z'
statusHistory:
- state: PENDING
  stateStartTime: '2025-02-22T10:39:19.874501Z'
- state: SETUP_DONE
  stateStartTime: '2025-02-22T10:39:19.910757Z'
- details: Agent reported job success
  state: RUNNING
  stateStartTime: '2025-02-22T10:39:20.284599Z'
yarnApplications:
- name: Spark Pi
  progress: 1.0
  state: FINISHED
  trackingUrl: http://google-cli-cluster-for-spark-m-local-10128-0-16-8088/proxy/application_1740218524166_0001/
(base) chimaonyekpere@MacBookPro ~ %

```

1.6.3 Step 3: Attempting to Increase Worker Nodes (Failed Due to SSD Quota)

After submitting the job, I attempted to **increase the number of worker nodes from 2 to 5**. However, the update **failed** due to an **SSD quota limitation**.

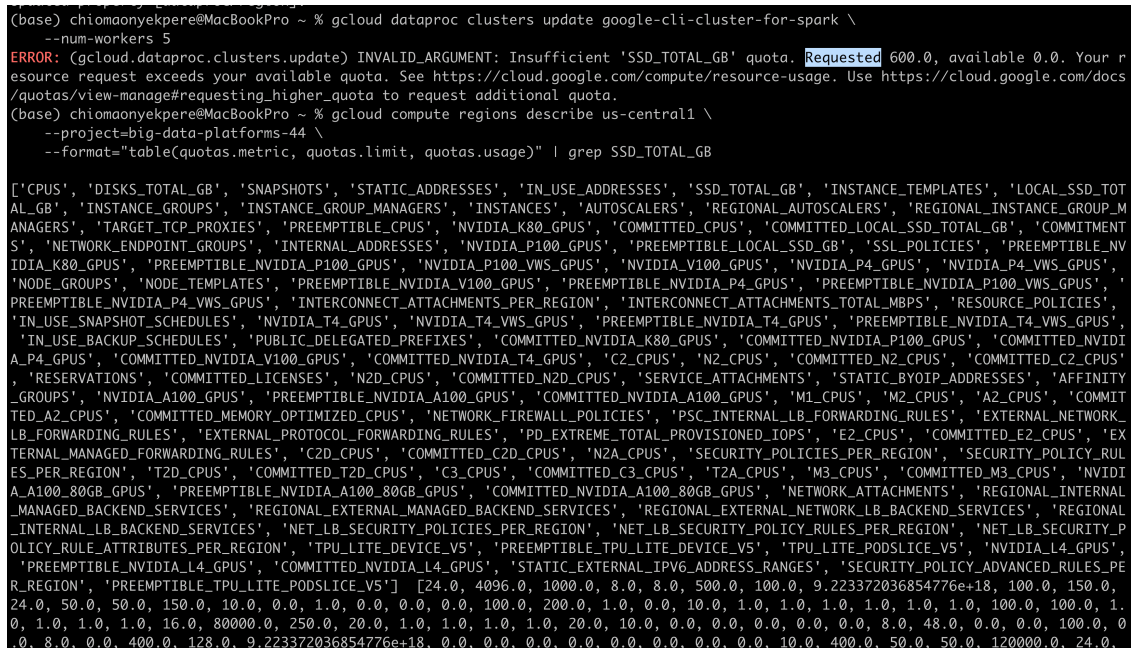
Command Used:

```
gcloud dataproc clusters update google-cli-cluster-for-spark \  
--num-workers 5
```

Error Received:

```
ERROR: (gcloud.dataproc.clusters.update) INVALID_ARGUMENT: Insufficient 'SSD_TOTAL_GB' quota.  
Requested 600.0, available 0.0. Your resource request exceeds your available quota.
```

[41]: `display(Image(filename="screenshots/cd20.png"))`



```
(base) chiamaonyekpere@MacBookPro ~ % gcloud dataproc clusters update google-cli-cluster-for-spark \  
--num-workers 5  
ERROR: (gcloud.dataproc.clusters.update) INVALID_ARGUMENT: Insufficient 'SSD_TOTAL_GB' quota. Requested 600.0, available 0.0. Your resource request exceeds your available quota. See https://cloud.google.com/compute/resource-usage. Use https://cloud.google.com/docs/quotas/view-manage#requesting_higher_quota to request additional quota.  
(base) chiamaonyekpere@MacBookPro ~ % gcloud compute regions describe us-central1 \  
--project=big-data-platforms-44 \  
--format="table(quotas.metric, quotas.limit, quotas.usage)" | grep SSD_TOTAL_GB  
['CPUS', 'DISKS_TOTAL_GB', 'SNAPSHOTS', 'STATIC_ADDRESSES', 'IN_USE_ADDRESSES', 'SSD_TOTAL_GB', 'INSTANCE_TEMPLATES', 'LOCAL_SSD_TOTAL_GB', 'INSTANCE_GROUPS', 'INSTANCE_GROUP_MANAGERS', 'INSTANCES', 'AUTOSCALERS', 'REGIONAL_AUTOSCALERS', 'REGIONAL_INSTANCE_GROUP_MANAGERS', 'TARGET_TCP_PROXIES', 'PREEMPTIBLE_CPUS', 'NVIDIA_K80_GPUS', 'COMMITTED_CPUS', 'COMMITTED_LOCAL_SSD_TOTAL_GB', 'COMMITMENTS', 'NETWORK_ENDPOINT_GROUPS', 'INTERNAL_ADDRESSES', 'NVIDIA_P100_GPUS', 'PREEMPTIBLE_LOCAL_SSD_GB', 'SSL_POLICIES', 'PREEMPTIBLE_NVIDIA_K80_GPUS', 'PREEMPTIBLE_NVIDIA_P100_GPUS', 'NVIDIA_P100_VWS_GPUS', 'NVIDIA_V100_GPUS', 'NVIDIA_P4_GPUS', 'NVIDIA_P4_VWS_GPUS', 'NODE_GROUPS', 'NODE_TEMPLATES', 'PREEMPTIBLE_NVIDIA_V100_GPUS', 'PREEMPTIBLE_NVIDIA_P4_GPUS', 'PREEMPTIBLE_NVIDIA_P100_VWS_GPUS', 'PREEMPTIBLE_NVIDIA_P4_VWS_GPUS', 'INTERCONNECT_ATTACHMENTS_PER_REGION', 'INTERCONNECT_ATTACHMENTS_TOTAL_MBPS', 'RESOURCE_POLICIES', 'IN_USE_SNAPSHOT_SCHEDULES', 'NVIDIA_T4_GPUS', 'NVIDIA_T4_VWS_GPUS', 'PREEMPTIBLE_NVIDIA_T4_GPUS', 'PREEMPTIBLE_NVIDIA_T4_VWS_GPUS', 'IN_USE_BACKUP_SCHEDULES', 'PUBLIC_DELEGATED_PREFIXES', 'COMMITTED_NVIDIA_K80_GPUS', 'COMMITTED_NVIDIA_P100_GPUS', 'COMMITTED_NVIDIA_P4_GPUS', 'COMMITTED_NVIDIA_V100_GPUS', 'COMMITTED_NVIDIA_T4_GPUS', 'C2_CPUS', 'N2_CPUS', 'COMMITTED_N2_CPUS', 'COMMITTED_C2_CPUS', 'RESERVATIONS', 'COMMITTED_LICENSES', 'N2D_CPUS', 'COMMITTED_N2D_CPUS', 'SERVICE_ATTACHMENTS', 'STATIC_BYOIP_ADDRESSES', 'AFFINITY_GROUPS', 'NVIDIA_A100_GPUS', 'PREEMPTIBLE_NVIDIA_A100_GPUS', 'COMMITTED_NVIDIA_A100_GPUS', 'M1_CPUS', 'M2_CPUS', 'A2_CPUS', 'COMMITTED_A2_CPUS', 'COMMITTED_MEMORY_OPTIMIZED_CPUS', 'NETWORK_FIREWALL_POLICIES', 'PSC_INTERNAL_LB_FORWARDING_RULES', 'EXTERNAL_NETWORK_LB_FORWARDING_RULES', 'EXTERNAL_PROTOCOL_FORWARDING_RULES', 'PD_EXTREME_TOTAL_PROVISIONED_IOPS', 'E2_CPUS', 'COMMITTED_E2_CPUS', 'EXTERNAL_MANAGED_FORWARDING_RULES', 'C2D_CPUS', 'COMMITTED_C2D_CPUS', 'N2A_CPUS', 'SECURITY_POLICIES_PER_REGION', 'SECURITY_POLICY_RULES_PER_REGION', 'T2D_CPUS', 'COMMITTED_T2D_CPUS', 'C3_CPUS', 'COMMITTED_C3_CPUS', 'T2A_CPUS', 'M3_CPUS', 'COMMITTED_M3_CPUS', 'NVIDIA_A100_80GB_GPUS', 'PREEMPTIBLE_NVIDIA_A100_80GB_GPUS', 'COMMITTED_NVIDIA_A100_80GB_GPUS', 'NETWORK_ATTACHMENTS', 'REGIONAL_INTERNAL_MANAGED_BACKEND_SERVICES', 'REGIONAL_EXTERNAL_MANAGED_BACKEND_SERVICES', 'REGIONAL_EXTERNAL_NETWORK_LB_BACKEND_SERVICES', 'REGIONAL_INTERNAL_LB_BACKEND_SERVICES', 'NET_LB_SECURITY_POLICIES_PER_REGION', 'NET_LB_SECURITY_POLICY_RULES_PER_REGION', 'NET_LB_SECURITY_POLICY_RULE_ATTRIBUTES_PER_REGION', 'TPU_LITE_DEVICE_VS', 'PREEMPTIBLE_TPU_LITE_DEVICE_VS', 'TPU_LITE_PODSLICE_VS', 'NVIDIA_L4_GPUS', 'PREEMPTIBLE_NVIDIA_L4_GPUS', 'COMMITTED_NVIDIA_L4_GPUS', 'STATIC_EXTERNAL_IPV6_ADDRESS_RANGES', 'SECURITY_POLICY_ADVANCED_RULES_PER_REGION', 'PREEMPTIBLE_TPU_LITE_PODSLICE_VS'] [24.0, 4096.0, 1000.0, 8.0, 8.0, 500.0, 100.0, 9.223372036854776e+18, 100.0, 150.0, 24.0, 50.0, 50.0, 150.0, 10.0, 0.0, 1.0, 0.0, 0.0, 0.0, 100.0, 200.0, 1.0, 0.0, 10.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 100.0, 100.0, 1.0, 1.0, 1.0, 1.0, 16.0, 80000.0, 250.0, 20.0, 1.0, 1.0, 1.0, 1.0, 20.0, 10.0, 0.0, 0.0, 0.0, 0.0, 0.0, 8.0, 48.0, 0.0, 0.0, 100.0, 0.0, 8.0, 0.0, 400.0, 128.0, 9.223372036854776e+18, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 10.0, 400.0, 50.0, 50.0, 120000.0, 24.0,
```

1.6.4 Step 4: Attempting to Increase SSD Quota (Failed)

To resolve the quota issue, I attempted to increase my SSD quota via gcloud.

Command Used:

```
gcloud compute project-info add-metadata \  
--project=big-data-platforms-44 \  
--metadata=quota-increase-request=SSD_TOTAL_GB,us-central1,1000
```

```
ERROR: (gcloud.compute.project-info.add-metadata) argument --metadata: Bad syntax for dict arg
```

```
[42]: display(Image(filename="screenshots/cd21.png"))
```

1.6.5 Step 5: Creating a New Cluster with Standard Persistent Disk (CLI)

Command Used:

18

Response: Cluster Created Successfully with Standard Disk (pd-standard)

```
[43]: display(Image(filename="screenshots/cd22.png"))
display(Image(filename="screenshots/cd24.png"))
```

```
--metadata=quota-increase-request=SSD_TOTAL_GB,us-central1,1000

ERROR: (gcloud.compute.project-info.add-metadata) argument --metadata: Bad syntax for dict arg: [us-central1]. Please see 'gcloud to
pic flags-file' or 'gcloud topic escaping' for information on providing list or dictionary flag values with special characters.
Usage: gcloud compute project-info add-metadata [optional flags]
optional flags may be --help | --metadata | --metadata-from-file

For detailed information on this command and its flags, run:
  gcloud compute project-info add-metadata --help
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters list --project=big-data-platforms-44 --region=us-central1

Listed 0 items.
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters create google-cli-cluster-for-spark \
--project=big-data-platforms-44 \
--region=us-central1 \
--zone=us-central1-f \
--master-machine-type=n1-standard-4 \
--master-boot-disk-type=pd-standard \
--master-boot-disk-size=100GB \
--num-workers=5 \
--worker-machine-type=n1-standard-2 \
--worker-boot-disk-type=pd-standard \
--worker-boot-disk-size=100GB \
--enable-component-gateway

Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/5acd35a8-1f43-378d-a9a3-7a01b9a0b09e].
Waiting for cluster creation operation...
WARNING: Consider using Auto Zone rather than selecting a zone manually. See https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/auto-zone
WARNING: Failed to validate permissions required for default service account: '801263886148-compute@developer.gserviceaccount.com'.
Cluster creation could still be successful if required permissions have been granted to the respective service accounts as mentioned
in the document https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/service-accounts#dataproc_service_accounts_2.
This could be due to Cloud Resource Manager API hasn't been enabled in your project '801263886148' before or it is disabled. Enable
it by visiting 'https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview?project=801263886148'.
```

The screenshot displays the Google Cloud IAM and admin console interface. The main content area is titled "Quotas and system limits for project 'big-data-platforms'". It includes a "MANAGE ALERT POLICIES" button and a "LEARN" link. A notification banner states: "Set up quota and system limit alerts. Get alerted if a quota is close to reaching its maximum. Click on more actions button ⓘ in a row to get started, or click 'Learn more' to view documentation." Below this, a status bar indicates "Values for quotas are being updated. This may take two to three weeks to complete." A summary box shows "Current usage > 90%" and "All quotas and system limits 19,354". A table lists various quotas with columns for Service, Name, Type, Dimensions, Value, Current usage percentage, and Current usage. The table includes entries for CPUs, VM instances, Networks, Persistent Disk Standard (GB), and Subnetwork ranges per network.

Service	Name	Type	Dimensions (e.g. location)	Value	Current usage percentage	Current usage
Compute Engine API	CPUs	Quota	region:us-central1	24	58.33%	
Compute Engine API	CPUs (all regions)	Quota		32	43.75%	
Compute Engine API	VM instances	Quota	region:us-central1	24	25%	
Compute Engine API	Networks	Quota		5	20%	
Compute Engine API	Persistent Disk Standard (GB)	Quota	region:us-central1	4,096 GB (4,096 TB)	9.77%	400
Compute Engine API	Subnetwork ranges per network	Quota	network_id: default	500	8.2%	

1.6.6 Step 6: Updating Worker Nodes via CLI

After creating the new cluster, I successfully increased the worker nodes from 2 to 8, then reduced it to 2.

Commands Used:

```
gcloud dataproc clusters update google-cli-cluster-for-spark --num-workers 8
```

Updated Worker Nodes from 5 → 8 Successfully

```
gcloud dataproc clusters update google-cli-cluster-for-spark --num-workers 2
```

Updated Worker Nodes from 8 → 2 Successfully

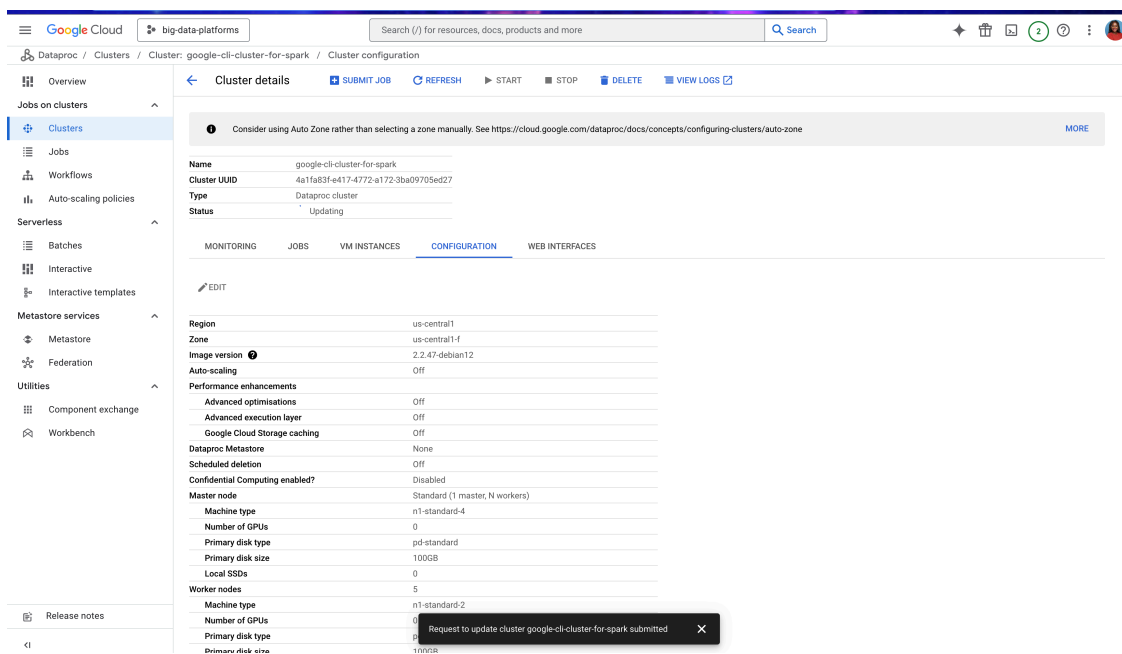
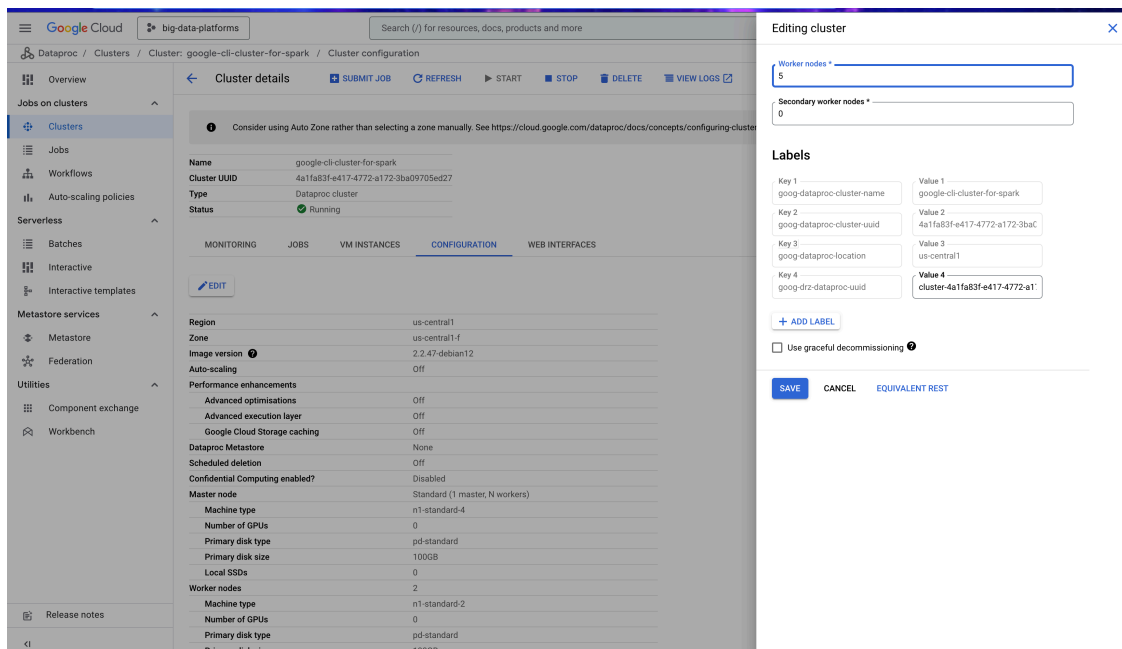
```
[44]: display(Image(filename="screenshots/cd23.png"))
```

```
Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/5acd35a8-1f43-378d-a9a3-7a01b9a0b09e].
Waiting for cluster creation operation...
WARNING: Consider using Auto Zone rather than selecting a zone manually. See https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/auto-zone
WARNING: Failed to validate permissions required for default service account: '801263886148-compute@developer.gserviceaccount.com'. Cluster creation could still be successful if required permissions have been granted to the respective service accounts as mentioned in the document https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/service-accounts#dataproc_service_accounts_2. This could be due to Cloud Resource Manager API hasn't been enabled in your project '801263886148' before or it is disabled. Enable it by visiting 'https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview?project=801263886148'.
WARNING: For PD-Standard without local SSDs, we strongly recommend provisioning 1TB or larger to ensure consistently high I/O performance. See https://cloud.google.com/compute/docs/disks/performance for information on disk I/O performance.
WARNING: The firewall rules for specified network or subnetwork would allow ingress traffic from 0.0.0.0/0, which could be a security risk.
WARNING: The specified custom staging bucket 'dataproc-staging-us-central1-801263886148-kg5dstgw' is not using uniform bucket level access IAM configuration. It is recommended to update bucket to enable the same. See https://cloud.google.com/storage/docs/uniform-bucket-level-access.
WARNING: No image specified. Using the default image version. It is recommended to select a specific image version in production, as the default image version may change at any time.
Waiting for cluster creation operation...done.
Created [https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark] Cluster placed in zone [us-central1-f].
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters update google-cli-cluster-for-spark \
--num-workers 8
Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/152059d0-264e-3661-9021-0f664d693e66].
Waiting for cluster update operation...done.
Updated [https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark].
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters update google-cli-cluster-for-spark \
--num-workers 2
Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/4b36952e-8d7a-397e-a75c-0b76396dd42e].
Waiting for cluster update operation...done.
Updated [https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark].
(base) chiomaonyekpere@MacBookPro ~ %
```

1.6.7 Step 7: Updating Worker Nodes via Google Cloud Console

Finally, I used Google Cloud Console to manually increase the worker nodes from 2 to 5.

```
[45]: display(Image(filename="screenshots/cd25.png"))
display(Image(filename="screenshots/cd26.png"))
```



1.6.8 Step 8: Deleting the Cluster (CLI)

Once all testing was complete, I deleted the Dataproc cluster via CLI.

Command Used:

```
gcloud dataproc clusters delete google-cli-cluster-for-spark --region=us-central1
```

Response: Cluster Deleted Successfully

The cluster was removed from Google Cloud.

[46]: `display(Image(filename="screenshots/cd27.png"))`

```
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters update google-cli-cluster-for-spark \
--num-workers 2
Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/4b36952e-8d7a-397e-a75c-0b76396dd42e].
Waiting for cluster update operation...done.
Updated [https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark
].
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters delete google-cli-cluster-for-spark \
--region-region
The cluster 'google-cli-cluster-for-spark' and all attached disks will be deleted.

Do you want to continue (Y/n)? y

ERROR: (gcloud.dataproc.clusters.delete) PERMISSION_DENIED: Permission denied on 'locations/region' (or it may not exist). This comm
and is authenticated as chioma.onyekpere@gmail.com which is the active account specified by the [core/account] property.
- '@type': type.googleapis.com/google.rpc.ErrorInfo
  domain: googleapis.com
  metadata:
    consumer: projects/big-data-platforms-44
    location: region
    service: dataproc.googleapis.com
  reason: LOCATION_POLICY_VIOLATED
- '@type': type.googleapis.com/google.rpc.LocalizedMessage
  locale: en-US
  message: Permission denied on 'locations/region' (or it may not exist).
(base) chiomaonyekpere@MacBookPro ~ % gcloud dataproc clusters delete google-cli-cluster-for-spark
The cluster 'google-cli-cluster-for-spark' and all attached disks will be deleted.

Do you want to continue (Y/n)? y

Waiting on operation [projects/big-data-platforms-44/regions/us-central1/operations/4fefeabf-a2b6-31f0-b3b1-a4b96448ccaf].
Waiting for cluster deletion operation...done.
Deleted [https://dataproc.googleapis.com/v1/projects/big-data-platforms-44/regions/us-central1/clusters/google-cli-cluster-for-spark
].
(base) chiomaonyekpere@MacBookPro ~ %
```

1.6.9 Summary of Task 2

Task	Status	Notes
Created Dataproc Cluster (SSD)	Success	Used <code>pd-ssd</code> , 2 workers
Submitted Spark Job	Success	Job completed without errors
Increased Worker Nodes (SSD)	Failed	Insufficient SSD quota (500GB limit)
Requested More SSD Quota	Not Allowed	Syntax error & quota limitation
Created Cluster with <code>pd-standard</code>	Success	Used standard persistent disks
Updated Worker Nodes to 8	Success	Increased via CLI
Updated Worker Nodes to 2	Success	Reduced via CLI
Updated Worker Nodes to 5 (Console)	Success	Modified via Google Console

Task	Status	Notes
Deleted Cluster	Success	Cluster removed via CLI

This experiment demonstrated how to **deploy, manage, and scale an Apache Spark cluster** using **Google Cloud CLI**. The key challenge was **insufficient SSD quota**, which required switching to **standard persistent disks**. The cluster was successfully scaled, and the project was completed with a full cycle of **creation, job execution, scaling, and deletion**.

1.7 Summary

This assignment involved setting up and managing an **Apache Spark Cluster on Google Cloud Dataproc** using both the **Google Cloud Console** and **Google Cloud SDK (gcloud CLI)**. The primary objectives were to: 1. **Create a Dataproc cluster**, 2. **Submit a Spark job** to run a sample computation, 3. **Verify job execution and results**, 4. **Scale the cluster by modifying the number of worker nodes**, and 5. **Clean up resources** after completion.

The assignment was performed in two phases: 1. **Google Cloud Console-based Approach** - Successfully created and managed a cluster via the web interface. 2. **Command Line Interface (CLI) Approach** - Replicated the same tasks using gcloud CLI commands.

During the CLI implementation, the initial **SSD-backed cluster** was created successfully, and a Spark job was submitted and executed without issues. However, an attempt to increase worker nodes **failed due to an insufficient SSD storage quota** (maximum limit: **500GB**). To resolve this, a new cluster was created with **standard persistent disks (pd-standard)** instead of SSDs. Further attempts to scale the cluster via CLI were successful—**worker nodes were increased to 8, then reduced to 2, and finally updated to 5 using the Cloud Console**.

1.8 Conclusion

This assignment provided hands-on experience in managing **Spark clusters on Google Cloud Dataproc**, demonstrating the differences between the **Google Cloud Console** and **CLI approaches**. Key takeaways include: - **The importance of resource quotas in cluster scaling**, particularly **SSD disk quotas** which limited expansion. - **The advantage of using persistent standard disks (pd-standard)** when SSD quotas are restrictive. - **Flexibility in cluster management**, where operations like scaling could be performed via both CLI and Console. - **The need to clean up resources** post-execution to avoid unnecessary costs.

Overall, the experiment was successful, with all required tasks completed despite encountering and resolving **storage quota limitations**.